

ПРАКТИЧЕСКАЯ РАБОТА 1: ПЕРВИЧНОЕ ВЕРИФИЦИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ «КАЛЬКУЛЯТОР»

Что такое тестирование?

Тестирование — это процесс проверки программы на соответствие ожиданиям. Ты не пишешь код, но проверяешь, правильно ли он работает.

Что такое баг (дефект)?

Это ошибка в программе, из-за которой она:

- считает неправильно,
- падает,
- ведёт себя странно,
- не реагирует на действия пользователя.

Что такое требования?

Это описание того, как программа должна работать. Если программа делает не то, что написано в требованиях — это баг.

Что такое верификация?

Это проверка: «Сделали ли мы программу правильно?»
(Сравниваем поведение программы с требованиями.)

Требования к калькулятору:

Калькулятор — это приложение для выполнения математических вычислений. Он должен быть простым, надёжным и корректно обрабатывать как обычные операции, так и ошибочные ситуации.

1. Ввод данных

- Пользователь может вводить цифры от 0 до 9 с помощью кнопок или клавиатуры.
- Допускается ввод одной десятичной точки (.) в рамках одного числа для работы с дробями (например: 3.14, 0.5, -2.7).
- Нельзя ввести более одной точки в одном числе (например, 3..14 — недопустимо).
- Число может начинаться с точки — в этом случае оно автоматически интерпретируется как 0.x (например, ввод .5 эквивалентен 0.5).
- Поддерживается ввод отрицательных чисел через операцию вычитания или при работе с результатами (например: $0 - 5 = \rightarrow -5$).

2. Арифметические операции

Калькулятор поддерживает четыре базовые арифметические операции:

- Сложение (+) — складывает два числа. Пример: $2 + 3 = \rightarrow$ результат 5.

- Вычитание (−) — вычитает второе число из первого. Пример: $10 - 4 = \rightarrow$ результат 6.
- Умножение (×) — перемножает два числа. Пример: $7 \times 8 = \rightarrow$ результат 56.
- Деление (÷) — делит первое число на второе. Пример: $15 \div 3 = \rightarrow$ результат 5.

Операции выполняются последовательно слева направо, без учёта математического приоритета (как в обычном бытовом калькуляторе).

Пример: $2 + 3 \times 4 = \rightarrow$ сначала $2 + 3 = 5$, затем $5 \times 4 = 20$.

3. Специальные функции

- Кнопка C (Clear) — полностью сбрасывает калькулятор: очищает текущий ввод, удаляет сохранённые значения и операции, возвращает дисплей к состоянию 0.
- Проценты (%) — преобразует текущее число в его сотую часть (то есть делит на 100).
Пример: ввести 200, нажать % $\rightarrow$ результат 2 (поскольку 1% от 200 = 2).
- Возведение в квадрат (x^2) — возводит текущее число в степень 2 (умножает число само на себя).
Пример: ввести 5, нажать $x^2 \rightarrow$ результат 25.
- Квадратный корень ($\sqrt{}$) — вычисляет неотрицательный корень квадратный из текущего числа.
Пример: ввести 16, нажать $\sqrt{} \rightarrow$ результат 4.
Если число отрицательное, операция должна быть заблокирована, а пользователю — показано сообщение об ошибке.

4. Обработка ошибок

- При попытке деления на ноль (например: $5 \div 0 =$) калькулятор не должен выдавать числовой результат. Вместо этого должно появиться понятное сообщение: *«Ошибка: деление на ноль невозможно!»*.
- При попытке извлечь квадратный корень из отрицательного числа должно появиться сообщение: *«Ошибка: нельзя извлечь корень из отрицательного числа»*.
- При вводе некорректных данных (например, попытка выполнить операцию с пустым полем) калькулятор должен либо игнорировать действие, либо показывать ошибку, но не завершать работу аварийно.

5. Отображение результата

- Результаты должны отображаться без лишних нулей в дробной части.
Примеры:
 - 4.0 $\rightarrow$ отображается как 4
 - 3.1400 $\rightarrow$ отображается как 3.14
- Если результат — целое число, он должен показываться без точки.
- Дисплей всегда должен содержать корректное числовое значение или сообщение об ошибке — пустой экран недопустим.

Задание: Прочитайте требования. Подумайте: какие действия вы бы проверили в первую очередь?

Вы — тестировщики в команде разработки.

Вам прислали две версии калькулятора:

1. Версия 1 (broken_calculator) — первая попытка сдать проект.
2. Версия 2 (good_calculator) — исправленная версия после ваших замечаний.

Ваша задача:

- Сначала протестировать Версию 1,
- Найти и описать все баги,
- Затем проверить Версию 2 — убедиться, что баги исправлены, и ничего нового не сломалось.

1. Скопируйте код из файла broken_calculator и вставьте его в проект в Pycharm.
2. Составьте «План проверки» (это ваши будущие тест-кейсы). Заполните таблицу исходя из требований до того, как начнёте нажимать кнопки:

№	Что проверяем?	Действия (какие кнопки нажимаем)	Что должно получиться?	Что получилось?
1	Сложение	$3 + 4 =$	7	
2	Деление на ноль	$5 \div 0 =$	Сообщение об ошибке	

3. Выполните проверку, теперь нажимайте кнопки и записывайте, что реально происходит, если нашли баг, то опишите его.

Пример:

Баг №1

Функция: Сложение

Шаги: Ввести $3 + 4 =$

Фактический результат: 12

Ожидаемый результат: 7

Вывод: Вместо сложения производится умножение.

4. Скопируйте код из файла good_calculator и вставьте его в проект в Pycharm.
5. Повторите те же проверки, что и для первой версии калькулятора. Проверьте, не появилось ли новых багов. Иногда, исправляя одно, ломают другое. Проверьте все функции снова, особенно те, что не были сломаны в Версии 1:
6. Составьте таблицу проверок.
7. Отчёт по тестированию калькулятора

Отчёт должен содержать:

1. Название работы
Например: «Тестирование калькулятора: Версия 1 и Версия 2»
2. Цель тестирования
(1–2 предложения: «Проверить соответствие калькулятора требованиям и найти ошибки»)
3. Описание проведённых проверок
 - Какие функции тестировались (сложение, проценты, деление на ноль и т.д.)
 - Какие действия выполнялись (можно кратко, без таблиц)
4. Результаты по Версии 1 (сломанной)
 - Сколько багов найдено
 - Краткое описание каждого бага (функция, что делали, что пошло не так)
5. Результаты по Версии 2 (исправленной)
 - Исправлены ли все найденные баги?
 - Появились ли новые ошибки?
6. Вывод

Реализуйте программы и напишите к ним тесты которые покрывают 100% кода.

Задача 1. Скидка на товар

Напиши программу, которая:

- Получает цену и процент скидки,
- Проверяет, что цена ≥ 0 и скидка от 0 до 100,
- Если всё ОК — считает итоговую цену,
- Если нет — выводит None.

Формула: цена * (1 - скидка / 100)

Задача 2. Надёжный пароль

Напиши программу, которая:

- Получает пароль (строку),
- Проверяет:
 - длина ≥ 8 ,
 - есть хотя бы одна цифра,
 - есть хотя бы одна заглавная буква,
- Выводит True, если всё ОК, иначе False.

